ClaimX — Repository Access

Important: The ClaimX source code is maintained in a **private GitHub repository**.

GitHub Repository
https://github.com/XI4122-bhupinderkaur/ClaimX

Because the repository is private, users who need to review or download the project must first submit a GitHub access request to the repository owner. Access must be granted before the code and project files can be cloned.

Once access is approved, the repository contains the project source code and the supporting project documentation available for this submission.

Access requirement: GitHub account + repository access approval.

ClaimX

MVP Mobile Claims Management Solution

Documentation + 5-Slide Executive Presentation + Demo Evidence

Technology

React Native • TypeScript • NestJS • Prisma • PostgreSQL • JWT

AI-assisted development

GitHub Copilot used for implementation, investigation, testing and documentation support.

Git repository

Prepared for MVP demonstration and organisational review

1. Executive Summary

ClaimX is an MVP mobile claims-management application designed to give a claim holder a single mobile experience for authentication, claim visibility, claim creation, claim details, fraud assessment visibility, payment visibility, notifications and account/settings access.

MVP objective

- Provide a demonstrable end-to-end mobile workflow rather than disconnected screens.
- Connect the React Native client to a local NestJS/Prisma/PostgreSQL backend.
- Secure protected APIs with JWT authentication and server-side ownership checks.
- Use GitHub Copilot to accelerate implementation, debugging, tests and API-contract analysis.

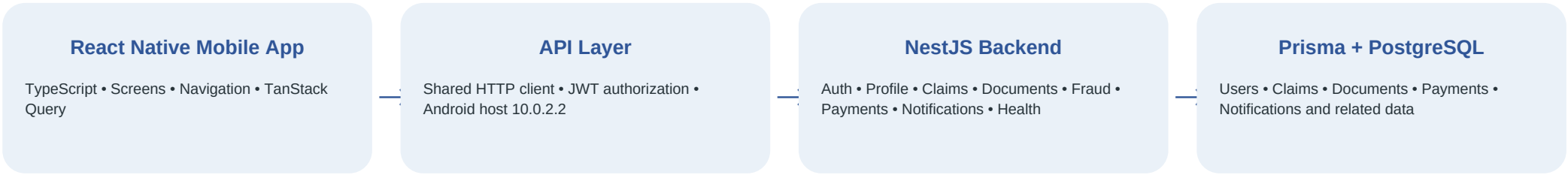
Current demonstrated flow

- Login — JWT authentication.
- Dashboard — claims, notifications and payments.
- Claims — list and claim details.
- Create Claim — enter claim information.
- Supporting views — notifications, payments and settings.

MVP positioning

- Focuses on core demonstrable claims-management capabilities.
- Production-grade document storage is outside scope.
- Forgot-password email delivery is outside scope.
- Full policy backend is outside scope.

2. Solution Architecture



Security and ownership

- Protected backend routes use JWT authentication.
- Server-side ownership checks prevent cross-user access to claims, payments and notifications.
- /auth/me returns a canonical profile shape without passwordHash or other sensitive credentials.
- Cross-user payment and notification access were verified as HTTP 403 during MVP testing.

3. MVP Feature Inventory

Capability	MVP status	Description
Authentication	Implemented	Login, JWT session persistence, current-us
Dashboard	Implemented	Overview, claims summary, recent claims, n
Claims	Implemented	Claim list, claim details and create-claim
Fraud assessment	Implemented	Claim detail exposes fraud score, risk lev
Payments	Implemented	Payment information and status visibility;
Notifications	Implemented	Notification listing and ownership enforce
Profile / Settings	Implemented	Account/profile APIs and settings experien
Documents	Metadata MVP	Document records can be created/listed/del
Policies	Not in MVP	Frontend placeholder/type exists; no Prism
Forgot Password	Not in MVP	UI placeholder only; reset-token/email wor

4. Backend API and Data Responsibilities

Core API areas

- Auth: POST /auth/login, GET /auth/me, POST /auth/logout.
- Claims: GET/POST/PATCH /claims... for create, list and management.
- Documents: claim document metadata lifecycle.
- Fraud: fraud assessment information for claims.
- Payments: payment records with ownership enforcement.
- Notifications: list/read data with ownership checks.
- Profile: canonical user profile retrieval/update.
- Health: GET /health for service availability.

Security validation performed

- Authenticated requests use Bearer JWT authorization.
- Cross-user payment access was rejected with HTTP 403.
- Cross-user notification access was rejected with HTTP 403.
- /auth/me was verified to return the authenticated user's canonical profile.
- PostgreSQL is the persistence layer through Prisma.
- No Prisma schema change was required for the demonstrated MVP flows.

5. Testing and Quality Evidence

Automated testing

- Frontend authentication-flow tests: 6 passed.
- Frontend API-client tests: 3 passed.
- Backend test suite: 9 suites and 38 tests passed.
- Backend build completed successfully.
- Prisma schema validation completed successfully.

Manual integration checks

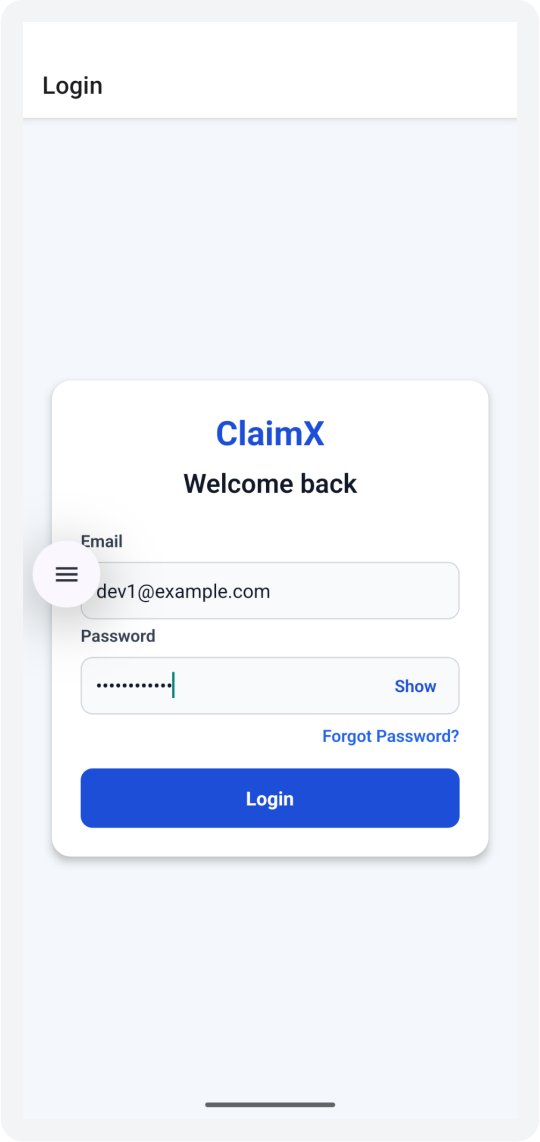
- GET /health returned HTTP 200 with {"status":"ok"}.
- Login with development user reached Dashboard.
- Dashboard loaded claims, notifications and payment information.
- Ownership and cross-user access protection were verified through API checks.

Demo environment

- React Native 0.87 / TypeScript.
- Node.js 18.20.8.
- NestJS + PostgreSQL + Prisma.
- Android Medium_Phone_API_36.1.
- Backend from emulator: http://10.0.2.2:3000.
- Metro: http://localhost:8081.

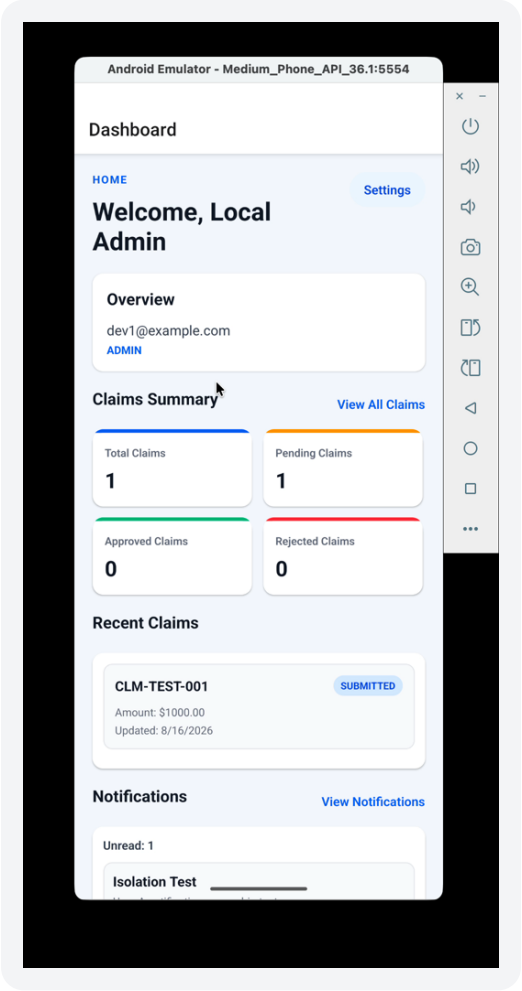
Additional Demo Evidence — Login

Login screen — development user credentials entered before authentication.



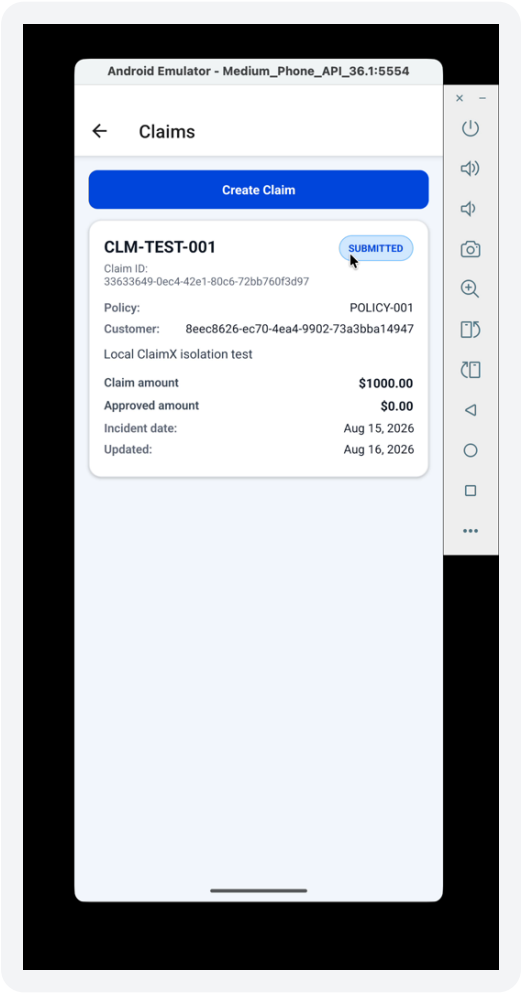
6. Demo Evidence — Dashboard

Dashboard — overview, claims summary, recent claims, notifications and quick actions.



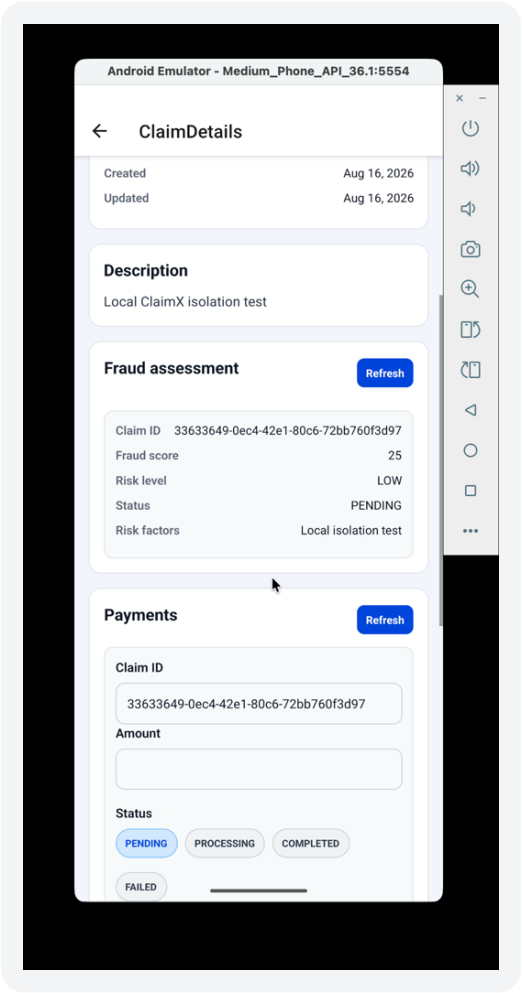
6. Demo Evidence — Claims

Claims — claim list with claim number, policy/customer information and status.



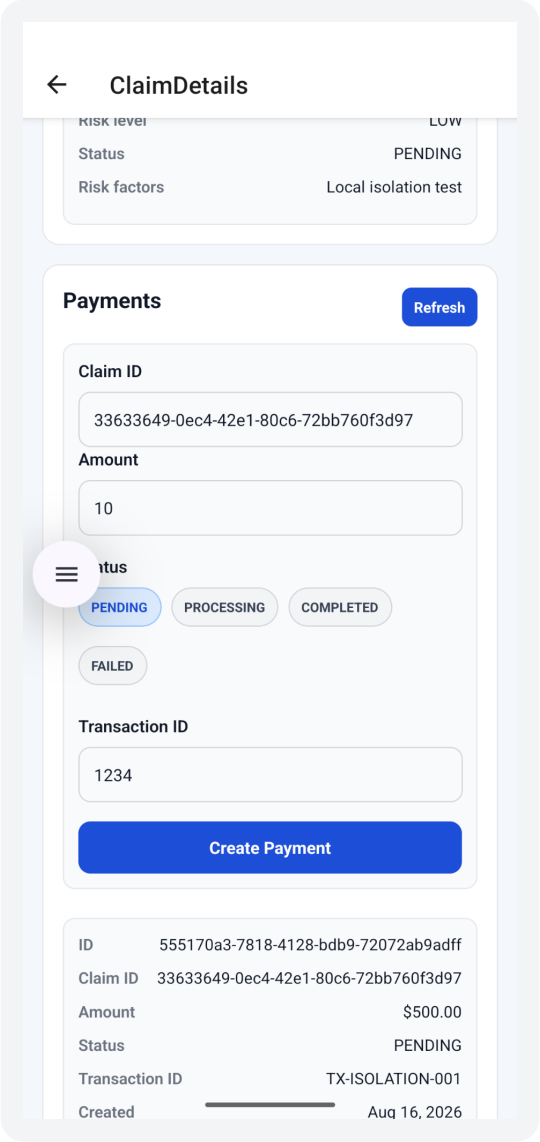
6. Demo Evidence — Claim Details

Claim Details — claim information plus fraud assessment and payment sections.



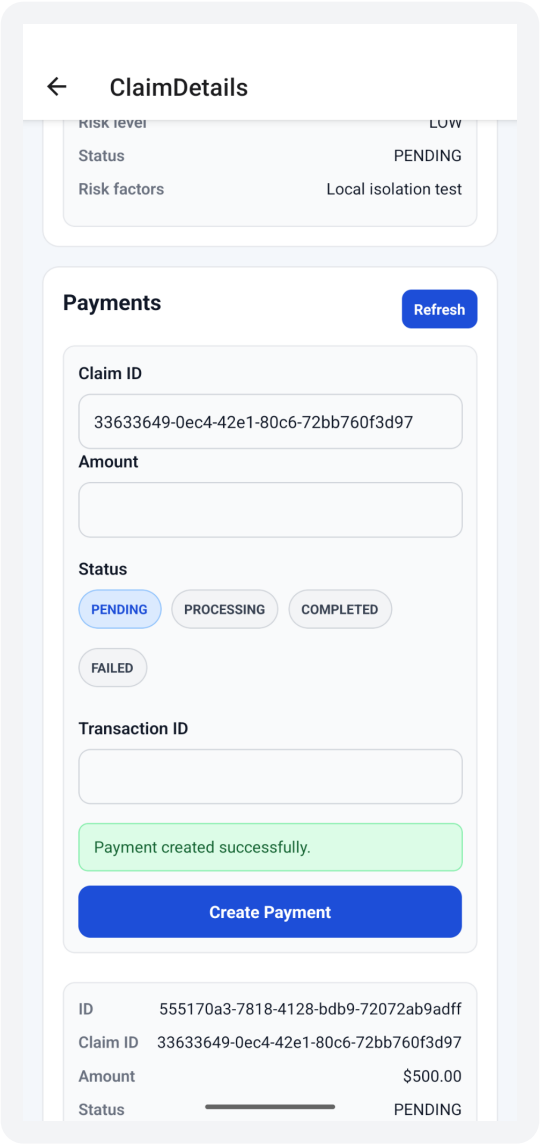
Additional Demo Evidence — Claim Details / Payment

Claim Details — payment section with claim ID, amount, status, transaction ID and Create Payment action.



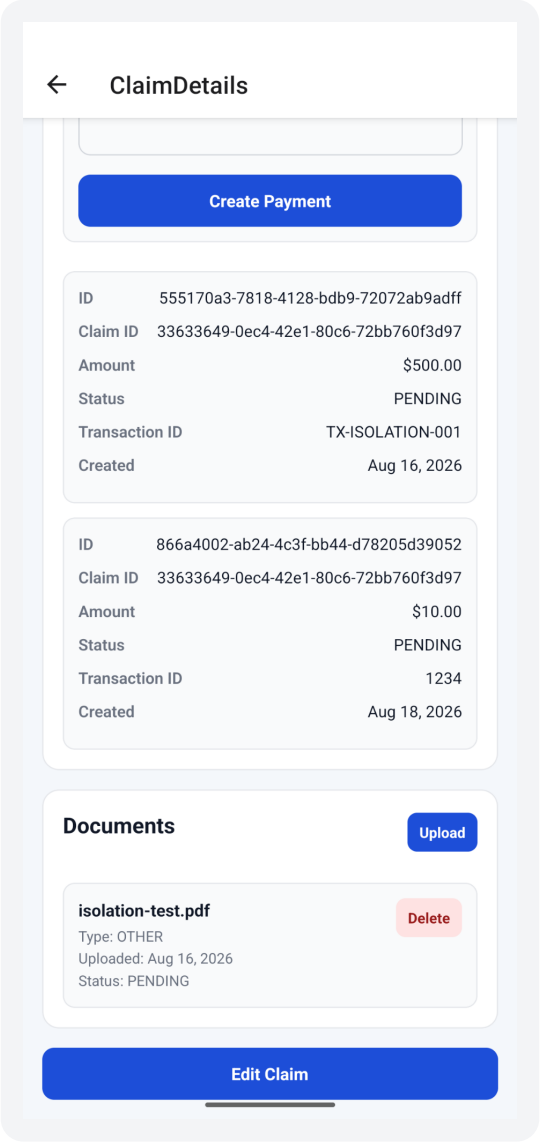
Additional Demo Evidence — Payment Created

Claim Details — successful payment creation confirmation and resulting payment record.



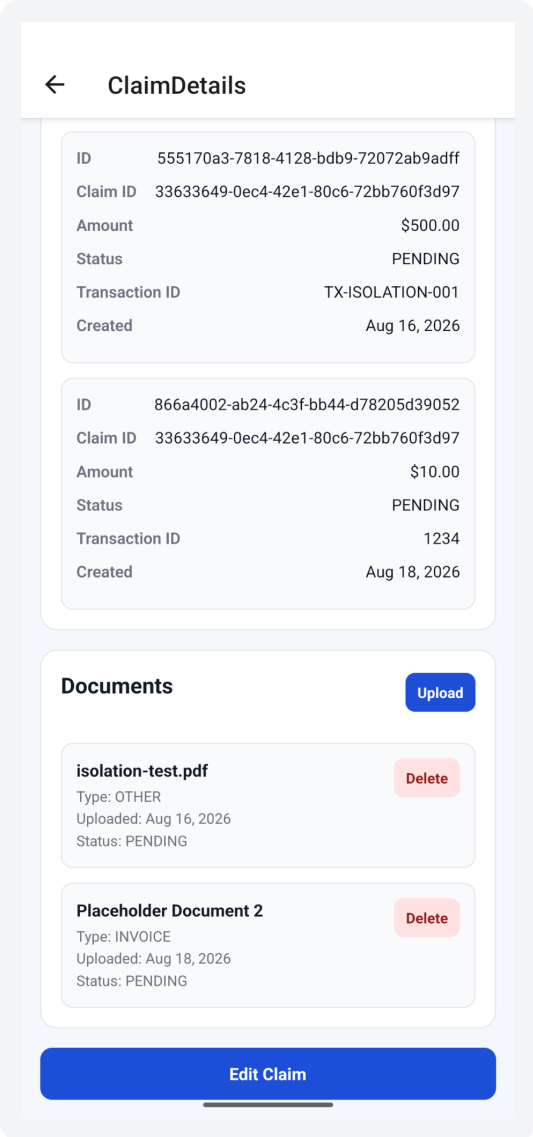
Additional Demo Evidence — Documents

Claim Details — document section showing the uploaded isolation-test.pdf document.



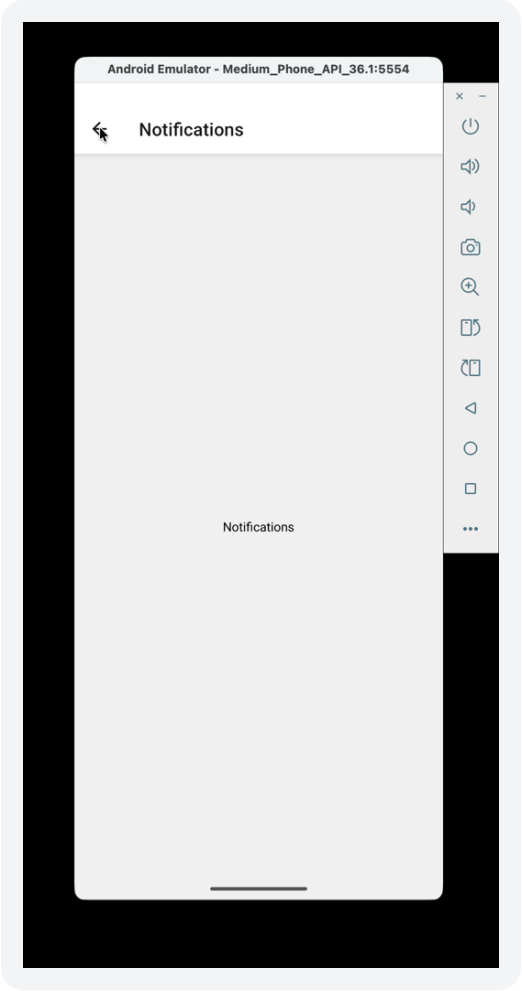
Additional Demo Evidence — Multiple Documents

Claim Details — document section showing multiple document records and upload/delete controls.



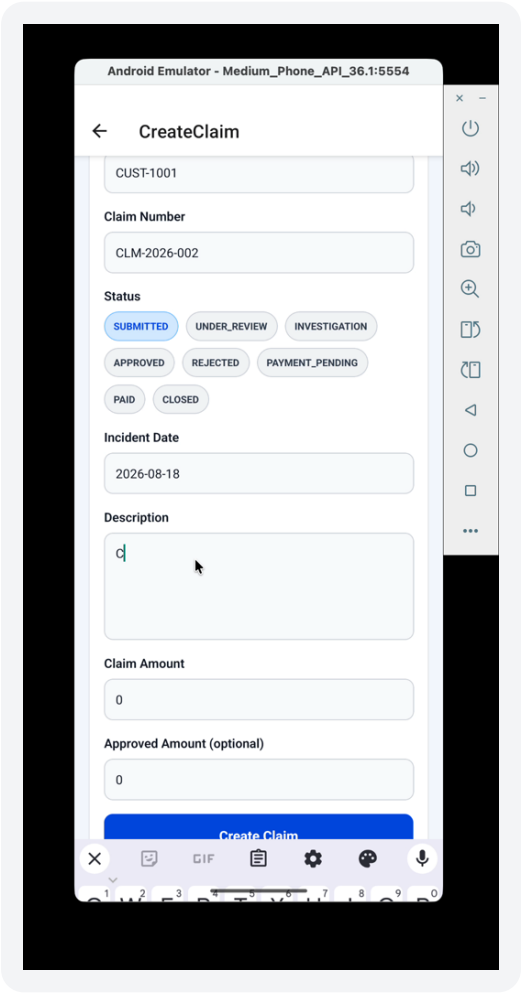
6. Demo Evidence — Notifications

Notifications — notifications view in the mobile application.



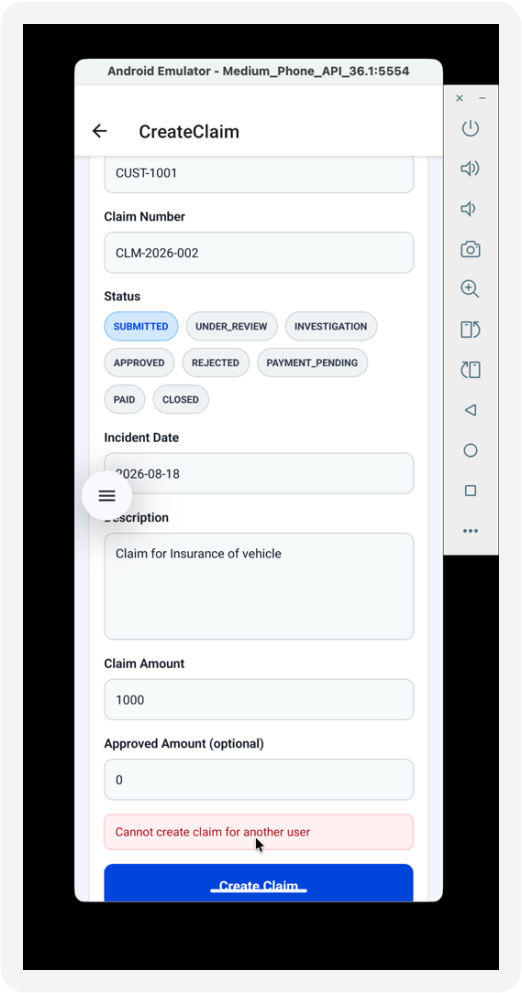
7. Demo Evidence — Create Claim

Create Claim — claim fields including policy/customer/claim number, status and incident date.



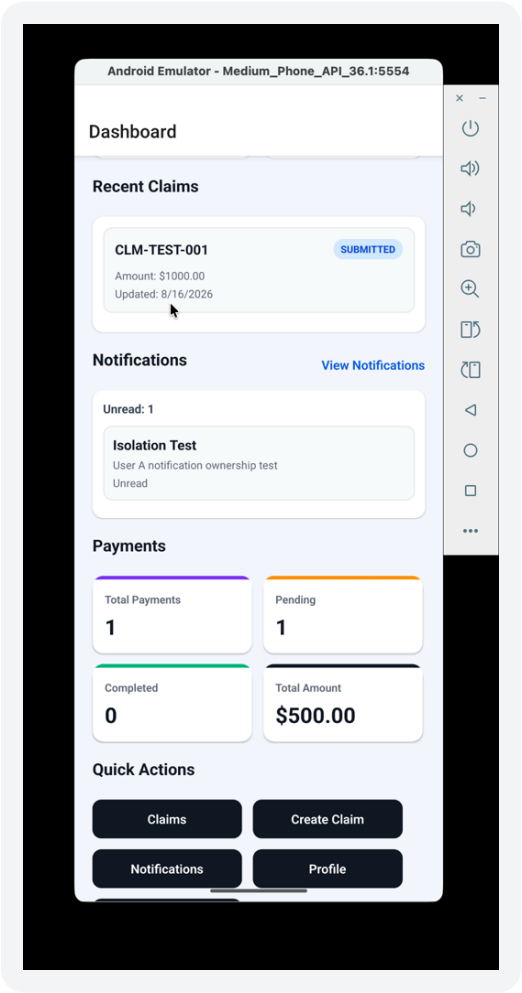
7. Demo Evidence — Create Claim Entry

Create Claim — populated claim description and amount fields before submission.



7. Demo Evidence — Result

Dashboard — recent claim and notification data visible after the workflow.



8. How to Run the ClaimX MVP

Prerequisites

- Node.js installed (demonstrated environment: 18.20.8).
- PostgreSQL running locally and accepting connections on 5432.
- Android Studio / Android SDK and emulator configured.
- Clone/download the project from the Git repository.

Start services

- cd ~/Desktop/ClaimX/backend
- npm run start:dev
- curl -i http://localhost:3000/health
- Expected: HTTP 200 and {"status":"ok"}.
- cd ~/Desktop/ClaimX
- npm start -- --reset-cache
- npm run android

Android networking

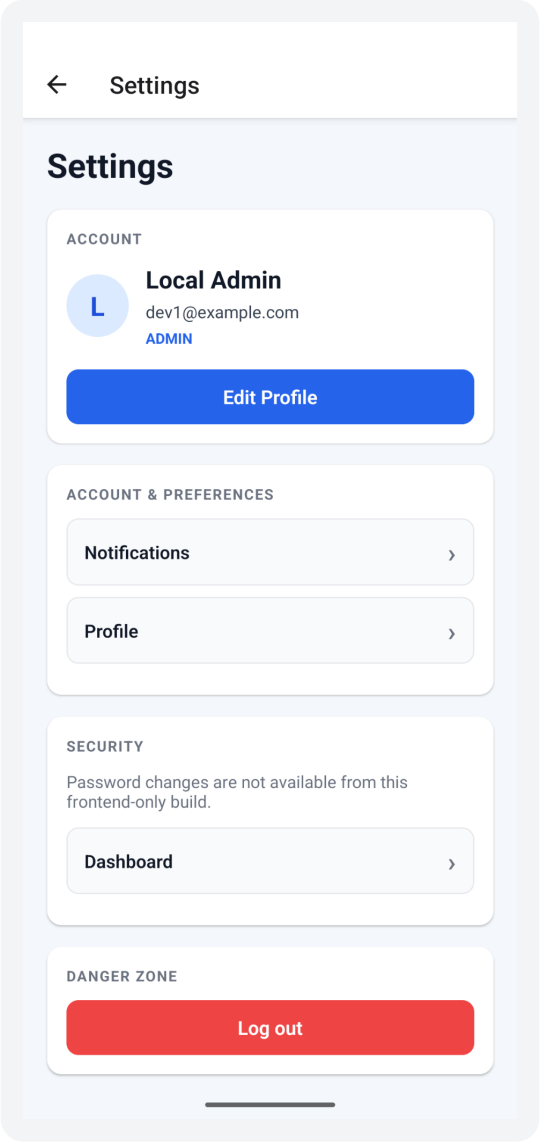
- Android emulator cannot use Mac localhost for the host backend.
- Use http://10.0.2.2:3000.
- The mobile API client falls back to this host when no API environment override is configured.
- Ensure PostgreSQL is running before starting backend.

9. Recommended MVP Demo Script

Scene	What to show
Scene 1 — Start	Show Android emulator and introduce ClaimX as a mobile claims-management MVP.
Scene 2 — Login	Enter dev1@example.com and the configured development password, then tap Login.
Scene 3 — Dashboard	Show overview, claims summary, recent claim, notifications and payments.
Scene 4 — Claims	Open Claims and show the claim list.
Scene 5 — Claim Details	Open a claim and show claim details, fraud assessment and payment information.
Scene 6 — Create Claim	Open Create Claim and demonstrate entering claim information.
Scene 7 — Notifications	Open Notifications and show notification data.
Scene 8 — Settings	Show Settings, profile/account options and logout as available in the current build.
Closing	State that complete source code, tests and project configuration are available in the Git repository.

Additional Demo Evidence — Settings

Settings — authenticated account information, preferences, security and logout controls.



10. MVP Scope, Limitations and Next Steps

Included in MVP

- Authenticated mobile claims experience.
- Dashboard with claims, notification and payment visibility.
- Claim list, claim detail and claim creation.
- Fraud assessment visibility on claim details.
- Ownership enforcement on protected resources.
- Local development and Android emulator demonstration.

Outside current MVP

- Production document binary upload/storage.
- Full policy domain and policy backend.
- Production forgot-password workflow and email delivery.
- Production object storage, malware scanning and signed URLs.
- Production deployment, observability, rate limiting and enterprise identity.

Future opportunities

- Secure object storage and authenticated/signed document access.
- Complete policy lifecycle and policy API.
- Secure password-reset email flow with expiring single-use tokens.
- Analytics, monitoring, audit trails and CI/CD.
- Approved AI capabilities for claim triage, document classification and fraud decision support.

11. Source Code, Repository and Handover

All ClaimX source code is maintained in the project Git repository. The repository is the authoritative source for downloading, building, testing and continuing development of the MVP.

Repository

- ClaimX Git repository.
- Use the repository to clone/download the complete project.
- Repository URL:
  https://github.com/XI4122-bhupinderkaur/ClaimX

Contains

- React Native mobile app.
- NestJS backend.
- Prisma schema.
- Frontend and backend tests.
- Configuration and project documentation.

Recommended structure

- src/ — React Native frontend.
- __tests__/ — frontend tests.
- android/ — Android project.
- backend/ — NestJS + Prisma.
- package.json and README.md.

Slide 1 — Context and Why Now

Challenges and Impact

Claims information is distributed across workflows and systems. Users and teams need a simpler mobile view of claim status, payments, notifications and supporting assessment information.

Strategic Imperative

A connected MVP creates a foundation for faster claim visibility, lower manual effort and a more consistent customer experience.

Shift in Operating Model

AI-assisted development and automation allow teams to move faster from requirement → implementation → test → demonstration while humans remain responsible for business decisions.

Why now: establish a working digital claims journey that can be demonstrated, validated and extended.

Slide 2 — Current Challenges and Gaps

Existing Process

Claim information must be surfaced across authentication, claims, payments, notifications and supporting assessment views.

Pain Points

Manual status checking, fragmented information, inconsistent visibility and slower handoffs can increase effort for users and support teams.

Technology Gaps

A connected mobile experience requires authenticated APIs, persistent data, ownership checks and a consistent client API layer.

MVP Response

ClaimX connects the mobile UI to a NestJS/Prisma/PostgreSQL backend and provides a single authenticated mobile workflow.

Slide 3 — Vision and Future State

Improved Efficiency

One mobile workflow exposes the information needed to understand a claim without moving between disconnected experiences.

Faster Decisions

Claim details combine operational information with fraud assessment and payment status for quicker review.

Better Experience

Customers get a clear dashboard, claim visibility, notifications and quick actions in one place.

Strategic Outcomes

Reduced manual effort, improved transparency, reusable API foundations and a scalable starting point for future automation.

Future operating model: people own decisions and exceptions; software automates retrieval, presentation and workflow; approved AI capabilities can support triage and analysis.

Slide 4 — Solution Overview

Mobile App

React Native + TypeScript: Login → Dashboard → Claims → Create Claim → Notifications → Payments → Settings.

API

Shared HTTP client with JWT Bearer authorization; Android emulator reaches local backend through 10.0.2.2:3000.

Backend

NestJS modules for Auth, Profile, Claims, Documents, Fraud, Payments, Notifications and Health.

Data

Prisma ORM + PostgreSQL with ownership checks and relational persistence.

AI Tool

GitHub Copilot assisted implementation, code inspection, API-contract comparison, debugging, test creation and focused fixes.

Human review remained responsible for scope, security decisions and final validation.

Slide 5 — Business Flow and Role of AI

1. Authenticate

User logs in and receives a JWT-backed session.

2. View

Dashboard retrieves claims, notifications and payment information.

3. Act

User opens a claim or creates a new claim.

4. Assess

Claim details expose fraud assessment and payment information.

AI intervention points

GitHub Copilot accelerates implementation, test generation, investigation and documentation.

Automation opportunities

Future opportunities include claim triage, document classification, anomaly detection and decision support.

Human decision

Business users retain responsibility for claim decisions, exceptions and final validation.

Outcome: connected mobile claims experience with an extensible technical foundation.

Repository: https://github.com/XI4122-bhupinderkaur/ClaimX

Appendix — MVP Acceptance Checklist

MVP readiness checklist

- Repository contains the complete ClaimX project.
- README contains setup and run instructions.
- Backend starts and /health returns 200.
- PostgreSQL is reachable on the configured development environment.
- Android emulator is available.
- User can log in successfully.
- Dashboard displays core claim/notification/payment information.
- Claims list and claim details are demonstrable.
- Create Claim flow is demonstrable.
- Ownership protections have been manually/API tested.
- Automated frontend/backend tests pass.
- Documentation and presentation are stored in the Git repository.

https://github.com/XI4122-bhupinderkaur/ClaimX